



ICR mini projet - Ransomware post quantique

08.06.2026

Camille Koestli

Table des matières

1. Consigne	3
2. Niveau de sécurité choisi	3
2.1. Choix des algorithmes	3
2.2. Paramètres cryptographiques	3
2.2.1. Clés symétriques (toutes à 256 bits)	3
2.2.2. Paramètres AES-256-GCM	3
2.2.3. Paramètres Argon2id	3
2.2.4. CRYSTALS-Kyber (ML-KEM-1024)	3
3. Architecture générale	4
3.1. Séparation client/serveur	4
3.2. Hiérarchie des clés	5
4. Description du ransomware	6
4.1. Fonctionnalités implémentées	6
4.2. Initialisation et chiffrement	7
4.2.1. Étape 1 : Initialisation du serveur	9
4.2.2. Étape 2 : Génération des clés cryptographiques (Client)	9
4.2.3. Étape 3 : Chiffrement récursif hiérarchique	9
4.2.4. Fichiers exclus du chiffrement	10
4.3. Déchiffrement d'un dossier complet ou d'un sous-dossier	11
4.3.1. Étape 1 : Récupération des credentials	11
4.3.2. Étape 2 : Récupération de la Root Key	11
4.3.3. Étape 3 : Déchiffrement récursif hiérarchique	12
4.4. Déchiffrement d'un fichier spécifique	13
4.4.1. Étape 1 : Localisation des fichiers clés	14
4.4.2. Étape 2 : Déchiffrement de la clé par le serveur	14
4.4.3. Étape 3 : Déchiffrement du fichier	14
4.5. Modification du mot de passe	15
4.5.1. Principe	15
4.5.2. Étape 1 : Préparation du nouveau mot de passe	16
4.5.3. Étape 2 : Réencapsulation de la Root Key	16
4.5.4. Étape 3 : Mise à jour des métadonnées	16
5. Implémentation technique	16
5.1. Architecture du code	16
5.1.1. Structure des fichiers	16
5.2. Bibliothèques cryptographiques utilisées	17
5.3. Choix d'implémentation	18
6. Tests et validation	18
6.1. Structure des tests	18
7. Fonctionnalités bonus	19
7.1. Chiffrement récursif hiérarchique de dossiers	19
7.1.1. Fonctionnement	19
7.1.2. Complexité	19
8. Améliorations possibles	20
9. Conclusion	20
10. Utilisation de l'IA	20

1. Consigne

Les consignes sont disponibles dans le document `mini_project_26.pdf`.

2. Niveau de sécurité choisi

Le système implémente un niveau de sécurité de 128 bits post-quantique, ce qui est le niveau 5 NIST. Cela permet une protection robuste contre les attaques classiques et quantiques.

2.1. Choix des algorithmes

AES-256-GCM pour le chiffrement symétrique des clés et des fichiers :

- Les clés sont de 256 bits ce qui offre 128 bits de sécurité post-quantique et donc est résistant à l'algorithme de Grover.
- L'algorithme garantit la confidentialité et l'authenticité des données.
- Les nonces sont à 96 bits et les tags d'authentification à 128 bits.

CRYSTALS-Kyber (ML-KEM-1024) pour pour la génération et l'encapsulation de clés post-quantiques :

- Il s'agit d'un algorithme post-quantique standardisé par le NIST.
- Le niveau 5 NIST offre 128 bits de sécurité post-quantique.
- Il résiste aux attaques par ordinateurs quantiques qui utilisent l'algorithme de Shor.

Argon2id pour la dérivation de clés à partir de mots de passe:

- Paramètres : `time_cost=2`, `memory_cost=64MB`, `parallelism=4`.
- Il est résistant aux attaques par GPU et bruteforce.
- Une dérivation de Master Key (MK) est réalisée à partir du mot de passe utilisateur.

2.2. Paramètres cryptographiques

2.2.1. Clés symétriques (toutes à 256 bits)

L'uniformité garantit une sécurité 128 bits post-quantique :

- **Clé de fichier (File Key)** : 256 bits (32 octets), une par fichier
- **Master Key (MK)** : 256 bits (32 octets), dérivée du mot de passe
- **Root Key (RK)** : 256 bits (32 octets), secret partagé Kyber

2.2.2. Paramètres AES-256-GCM

- **Nonce** : 96 bits (12 octets), qui est la taille optimale pour GCM
- **Tag d'authentification** : 128 bits (16 octets)
- **Taille de bloc** : 128 bits

2.2.3. Paramètres Argon2id

- **Salt** : 128 bits (16 octets)
- **Hash de sortie** : 256 bits (32 octets)
- **time_cost** : 2 itérations
- **memory_cost** : 65536 KB
- **parallelism** : 4 threads

2.2.4. CRYSTALS-Kyber (ML-KEM-1024)

- **Clé publique** : 1568 octets
- **Clé secrète** : 3168 octets
- **Ciphertext** : 1568 octets
- **Secret partagé** : 256 bits (32 octets), qui devient la Root Key

- **Niveau NIST** : 5 (équivalent AES-256 post-quantique)

Tous ces paramètres sont définis dans `config.py`.

3. Architecture générale

3.1. Séparation client/serveur

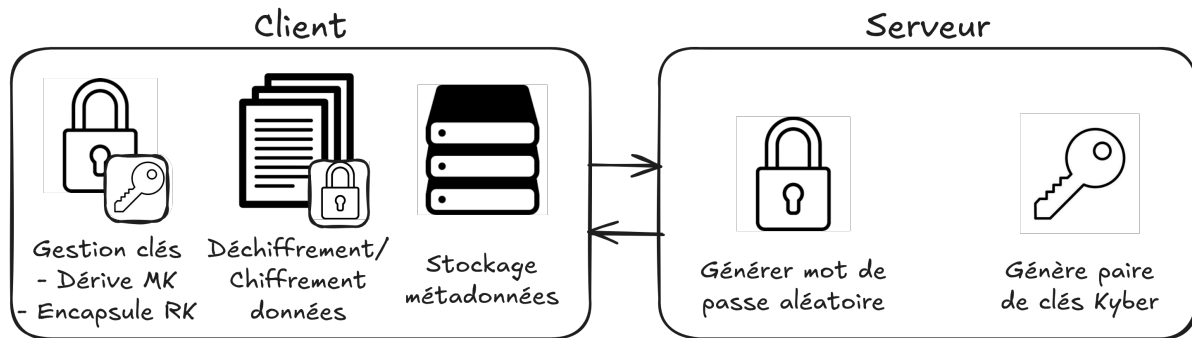


Fig. 1. – Architecture client-serveur du ransomware

Le ransomware est composé de deux parties distinctes :

Serveur (attaquant) :

- Génère la paire de clés CRYSTALS-Kyber (publique/secrète)
- Génère le mot de passe aléatoire mémorable
- Conserve la clé secrète Kyber en mémoire sécurisée
- Fournit les credentials pour le déchiffrement

Client (victime) :

- Dérive la Master Key (MK) à partir du mot de passe
- Encapsule la Root Key (RK) via Kyber
- Chiffre/déchiffre les fichiers de manière hiérarchique et récursive
- Stocke les métadonnées localement (`rootkey.bin`, fichiers `.key`, `.root_key`, `.enc`)

La communication entre le client et le serveur est réalisée localement grâce à des appels de fonctions. Pour un ransomware réel, cette communication serait effectuée grâce à un canal réseau sécurisé (ex: HTTPS).

3.2. Hiérarchie des clés

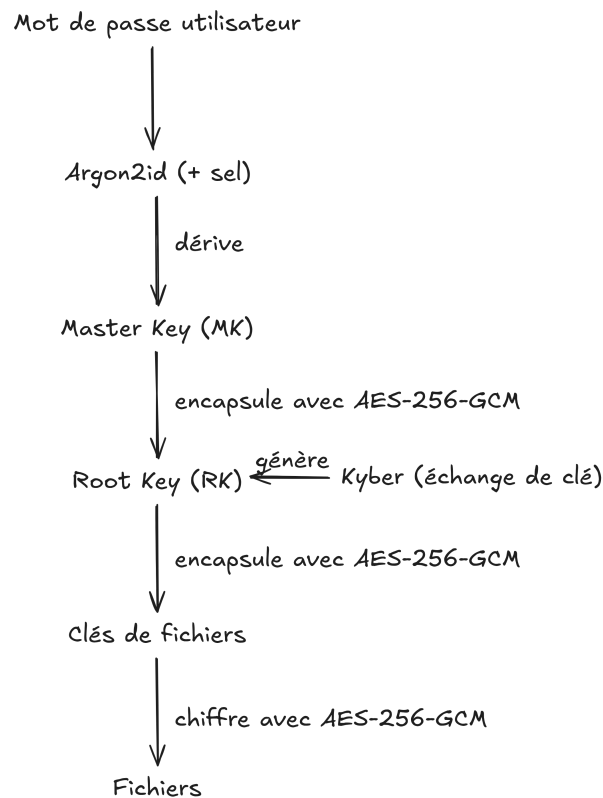


Fig. 2. – Hiérarchie cryptographique des clés

Le système utilise une architecture hiérarchique à 4 niveaux :

Master Key (MK) : Elle est dérivée du mot de passe utilisateur avec Argon2id + sel. Elle permet de protéger la Root Key et peut être changée sans rechiffrer les fichiers.

Root Key (RK) : Elle est générée par Kyber (secret partagé de 256 bits). Elle est encapsulée avec la MK (AES-256-GCM) et constitue la racine de la hiérarchie de chiffrement. Elle est stockée dans `rootkey.bin`.

Clés de dossiers (folder_key) : Une clé unique de 256 bits est générée pour chaque sous-dossier. Elle est chiffrée avec la clé de son dossier parent (ou la RK pour le niveau racine) et stockée dans un fichier `nomDossier.key`. Pour les dossiers imbriqués, une copie chiffrée avec la RK est également stockée dans `nomDossier.root_key`, permettant le déchiffrement individuel via le serveur.

Clés de fichiers (file_key) : Une clé unique de 256 bits est générée par fichier. Elle est chiffrée avec la `folder_key` du dossier parent (ou la RK si au niveau racine) et stockée dans `nomFichier.key`. Pour les fichiers dans des sous-dossiers, une copie chiffrée avec la RK est également stockée dans `nomFichier.root_key`.

Cette hiérarchie permet :

- Changement de mot de passe sans rechiffrement des fichiers
- Déchiffrement complet récursif via la hiérarchie parent → enfant
- Déchiffrement individuel d'un fichier ou dossier via le serveur (en utilisant le `.root_key`)

4. Description du ransomware

4.1. Fonctionnalités implémentées

Le ransomware possède les fonctionnalités suivantes :

Chiffrement des fichiers : Le système permet le chiffrement récursif et hiérarchique d'un dossier complet. Un mot de passe de 8 à 15 caractères est généré automatiquement depuis la wordlist `rockyou.txt`. Chaque fichier est chiffré avec une `file_key` unique de 256 bits, stocké sous forme de `fichier.enc`. Chaque dossier dispose d'une `folder_key` propre. Les clés sont chaînées hiérarchiquement : la clé d'un item est chiffrée avec la clé de son parent (`.key`). Pour les items hors du niveau racine, une copie supplémentaire de la clé est chiffrée avec la Root Key et stockée en `.root_key`, permettant le déchiffrement individuel.

Déchiffrement des fichiers : Deux méthodes de déchiffrement sont disponibles dans le menu principal. La première méthode, `decrypt_all()`, effectue un déchiffrement récursif complet en utilisant la hiérarchie des clés ($RK \rightarrow folder_key \rightarrow file_key$). La seconde méthode, `decrypt_file()`, permet le déchiffrement individuel d'un seul fichier : le client envoie le fichier `.root_key` au serveur, qui le déchiffre avec la Root Key (via Kyber) et retourne la `file_key` en clair.

Gestion du mot de passe : Cette fonction permet de changer le mot de passe sans avoir besoin de rechiffrer tous les fichiers. La méthode `change_password()` génère un nouveau mot de passe aléatoire et réencapsule uniquement la Root Key avec la nouvelle Master Key dérivée. Cette optimisation évite une opération coûteuse de rechiffrement complet qui pourrait prendre du temps selon la quantité de données à chiffrer.

4.2. Initialisation et chiffrement

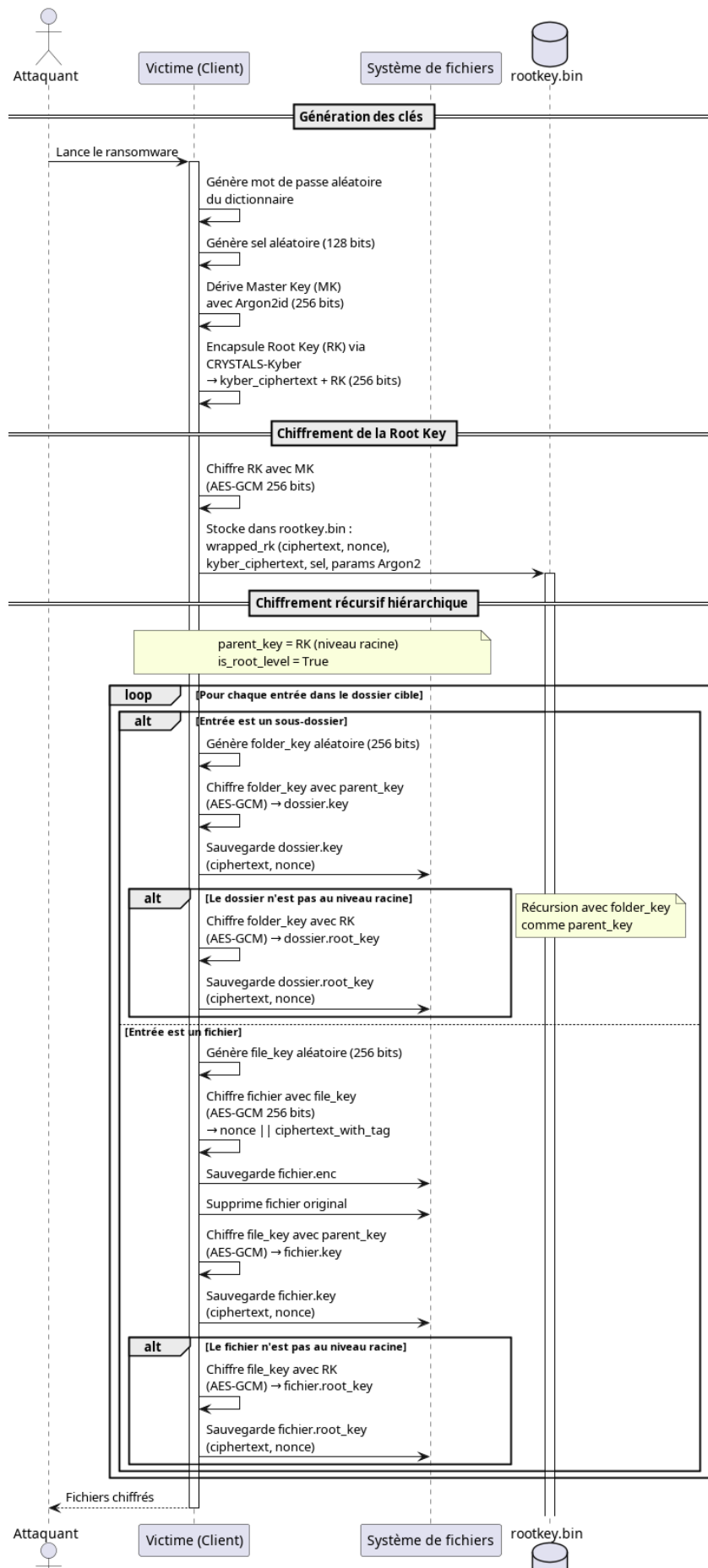


Fig. 3. – Processus d'initialisation et de chiffrement

4.2.1. Étape 1 : Initialisation du serveur

Le serveur (`RansomwareServer.initialize_server()`) va :

1. Générer d'une paire de clés CRYSTALS-Kyber (publique/secrète).
2. Générer un mot de passe aléatoire grâce à `wordlist.generate_random_password()`. Le mot de passe est un seul mot de 8 à 15 caractères conformément à la consigne.
 - Exemple : "password123" (11 caractères)
 - Les mots sont filtrés de la liste `rockyou.txt` (8-15 caractères)
3. Générer d'un sel aléatoire de 128 bits.
4. Préparer des paramètres Argon2 (`time_cost=2, memory_cost=64MB, parallelism=4`).
5. Retourner au client : password, salt, paramètres Argon2, clé publique Kyber.

4.2.2. Étape 2 : Génération des clés cryptographiques (Client)

Le client (`RansomwareClient.encrypt_directory()`) va :

1. **Dériver la Master Key (MK) :**
 - Utilise Argon2id avec le password et le sel reçus. Cela produit une clé de 256 bits.
2. **Générer la Root Key (RK) :**
 - Encapsulation Kyber avec la clé publique du serveur. Cela produit un ciphertext Kyber (1568 octets) et un secret partagé (256 bits).
 - Le secret partagé devient la Root Key.
3. **Protéger de la Root Key :**
 - Encapsulation de la RK avec la MK avec AES-GCM. Cela produit un ciphertext incluant le tag (16 octets en suffixe) et un nonce (96 bits).
 - On sauvegarde dans `rootkey.bin` avec un format JSON/base64 :

```
{
  "wrapped_rk_ciphertext": "...",
  "wrapped_rk_nonce": "...",
  "kyber_ciphertext": "...",
  "salt": "...",
  "argon2_params": {...}
}
```

4.2.3. Étape 3 : Chiffrement récursif hiérarchique

Le chiffrement est réalisé par `_encrypt_dir_recursive()` qui traite chaque entrée du dossier courant avec la `parent_key` du niveau courant (RK pour la racine, `folder_key` sinon).

Pour chaque sous-dossier :

1. **Générer une `folder_key` :**
 - Clé aléatoire de 256 bits avec `secrets.token_bytes(32)`.
2. **Sauvegarder `dossier.key` :**
 - La `folder_key` est chiffrée avec `parent_key` (AES-GCM) et stockée dans `dossier.key` (JSON/base64 : ciphertext, nonce).
3. **Sauvegarder `dossier.root_key` (si hors niveau racine) :**
 - La `folder_key` est aussi chiffrée avec la RK et stockée dans `dossier.root_key`. Cela permettra le déchiffrement individuel via le serveur.
4. **Récursion :** le sous-dossier est traité récursivement avec `folder_key` comme `parent_key`.

Pour chaque fichier :

1. Générer une `file_key` :

- Clé aléatoire de 256 bits.

2. Chiffrer le contenu :

- AES-256-GCM produit `nonce || ciphertext_with_tag` (tag de 16 octets en suffixe). Le résultat est sauvegardé dans `fichier.ext.enc`, puis le fichier original est supprimé.

3. Sauvegarder `fichier.ext.key` :

- La `file_key` est chiffrée avec `parent_key` (AES-GCM) et stockée en JSON/base64.

4. Sauvegarder `fichier.ext.root_key` (si hors niveau racine) :

- La `file_key` est aussi chiffrée avec la RK et stockée pour permettre le déchiffrement individuel.

Exemple de structure après chiffrement d'un dossier `data/` contenant `file.txt` et un sous-dossier `sub/file2.txt` :

```
data/
  file.txt.enc      ← contenu chiffré (nonce||ciphertext)
  file.txt.key      ← file_key chiffrée avec RK
  sub.key           ← folder_key de sub/ chiffrée avec RK
  sub/
    file2.txt.enc   ← contenu chiffré
    file2.txt.key   ← file_key chiffrée avec folder_key(sub)
    file2.txt.root_key ← file_key chiffrée avec RK
  rootkey.bin       ← RK encapsulée avec MK
```

4.2.4. Fichiers exclus du chiffrement

L'implémentation a été conçue pour exclure certains fichiers et dossiers afin de préserver le code source du ransomware et les dossiers système. Les exclusions sont définies dans des listes dans `config.py` :

- Fichiers Python (`.py`, `.pyc`)
- Le code source du ransomware (`client.py`, `server.py`, ...)
- Les fichiers de clés (`.key`, `.root_key`)
- Les fichiers chiffrés (`.enc`)
- Le fichier `rootkey.bin`
- Dossiers système (`.git`, `__pycache__`, `.venv`)

4.3. Déchiffrement d'un dossier complet ou d'un sous-dossier

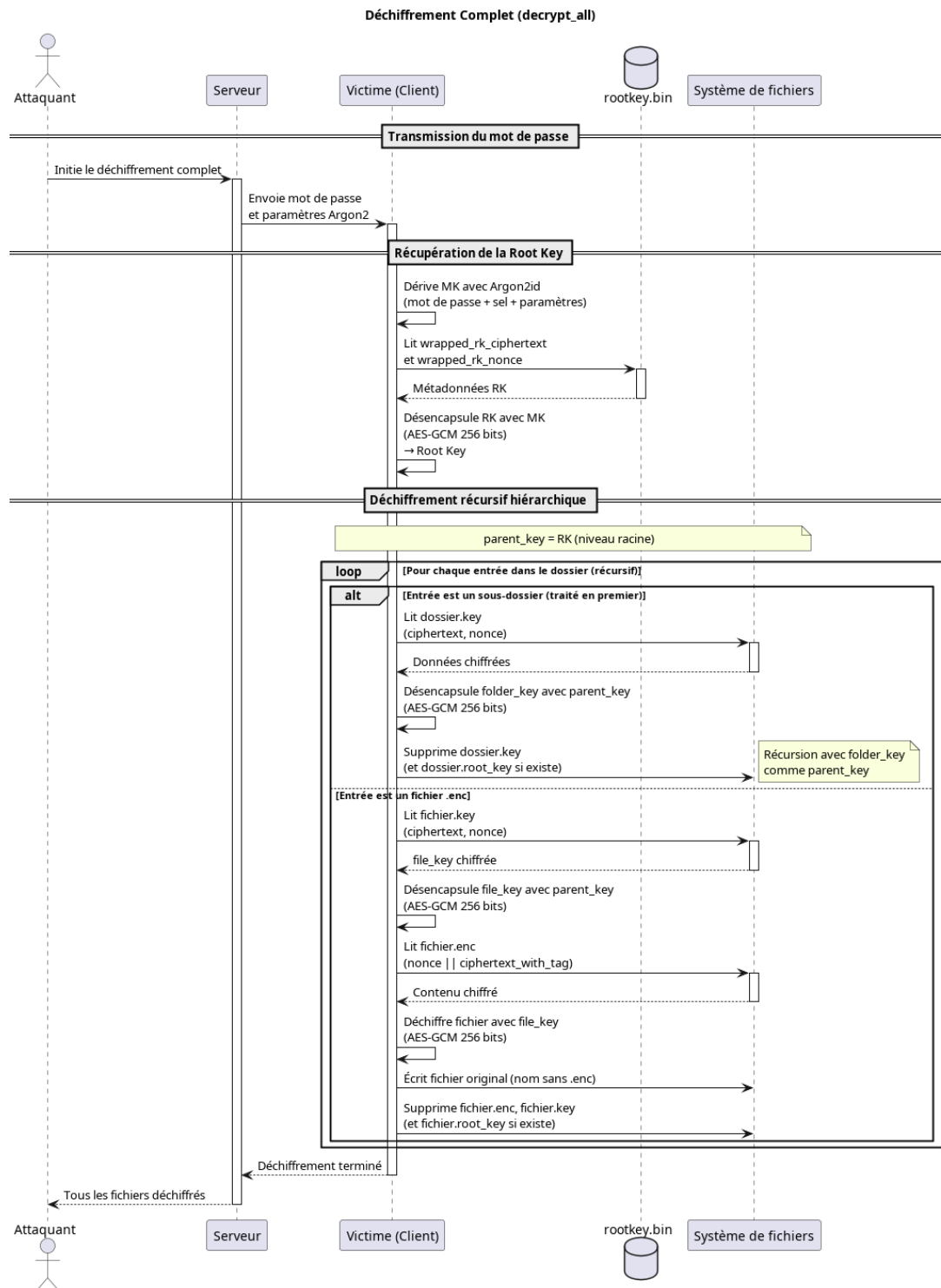


Fig. 4. – Processus de déchiffrement complet avec le mot de passe avec la méthode `decrypt_all()`

4.3.1. Étape 1 : Récupération des credentials

1. Le client demande au serveur les credentials complets.
2. Le serveur retourne : password, salt, paramètres Argon2.

4.3.2. Étape 2 : Récupération de la Root Key

Pour récupérer la Root Key, le client va :

1. Dériver la Master Key :

- Utilise Argon2id avec le password, le salt et les paramètres.
- On recalcule la même MK que lors du chiffrement.

2. Lire rootkey.bin :

- Charge wrapped_rk_ciphertext et wrapped_rk_nonce.

3. Désencapsuler la Root Key :

- Utilise AES-GCM avec la MK. La sortie est la Root Key.

4.3.3. Étape 3 : Déchiffrement récursif hiérarchique

Le déchiffrement est réalisé par `_decrypt_dir_recursive()` en partant de la racine avec RK comme `parent_key`.

Pour chaque sous-dossier (traité en premier) :

1. **Lire dossier.key** et désencapsuler la `folder_key` avec `parent_key` (AES-GCM).
2. Supprimer `dossier.key` et `dossier.root_key` s'il existe.
3. Récurser dans le sous-dossier avec `folder_key` comme `parent_key`.

Pour chaque fichier .enc :

1. **Lire fichier.key** et désencapsuler la `file_key` avec `parent_key` (AES-GCM).
2. **Lire fichier.enc** : extraire le nonce (12 premiers octets) et le ciphertext avec tag.
3. **Déchiffrer** avec AES-GCM en utilisant `file_key` et nonce.
4. **Écrire** le fichier original (nom sans .enc).
5. **Supprimer** `fichier.enc`, `fichier.key` et `fichier.root_key` s'il existe.

4.4. Déchiffrement d'un fichier spécifique

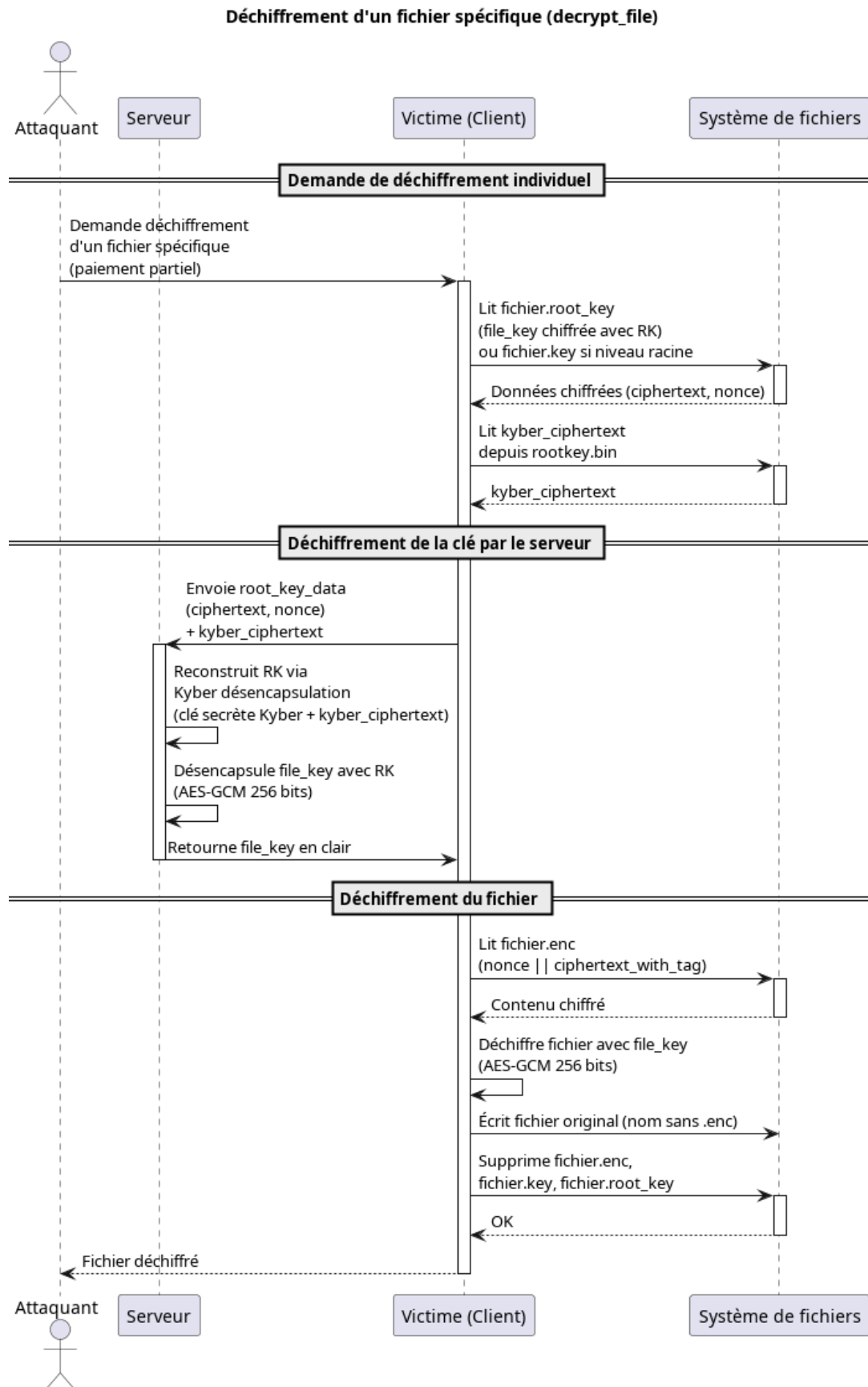


Fig. 5. – Déchiffrement individuel d'un fichier via le serveur avec la méthode `decrypt_file()`

Le déchiffrement individuel ne nécessite pas le mot de passe complet. La victim envoie le fichier `.root_key` au serveur, qui le déchiffre avec la Root Key et retourne la `file_key`.

4.4.1. Étape 1 : Localisation des fichiers clés

1. Le client identifie le fichier `.enc` à déchiffrer.
2. Si un `.root_key` existe pour ce fichier (item hors-racine), il est utilisé. Sinon, le `.key` est utilisé directement (le fichier est au niveau racine, donc sa clé est déjà chiffrée avec RK).
3. Le `kyber_ciphertext` est lu depuis `rootkey.bin`.

4.4.2. Étape 2 : Déchiffrement de la clé par le serveur

1. Le client envoie au serveur :
 - Le contenu du fichier `.root_key` (ou `.key` si racine) : `{ciphertext, nonce}` en base64.
 - Le `kyber_ciphertext` pour que le serveur puisse reconstruire la Root Key.
2. Le serveur (`decrypt_file_key()`) :
 - Reconstitue la Root Key via Kyber désencapsulation (`kyber_decapsulate(secret_key, kyber_ciphertext)`).
 - Désencapsule la `file_key` avec la Root Key (AES-GCM).
 - Retourne la `file_key` en clair au client.

4.4.3. Étape 3 : Déchiffrement du fichier

1. **Lire `fichier.enc`** : extraire le nonce (12 premiers octets) et le ciphertext avec tag.
2. **Déchiffrer** avec AES-GCM en utilisant la `file_key` reçue du serveur.
3. **Écrire** le fichier original (nom sans `.enc`).
4. **Supprimer** `fichier.enc`, `fichier.key` et `fichier.root_key`.

4.5. Modification du mot de passe

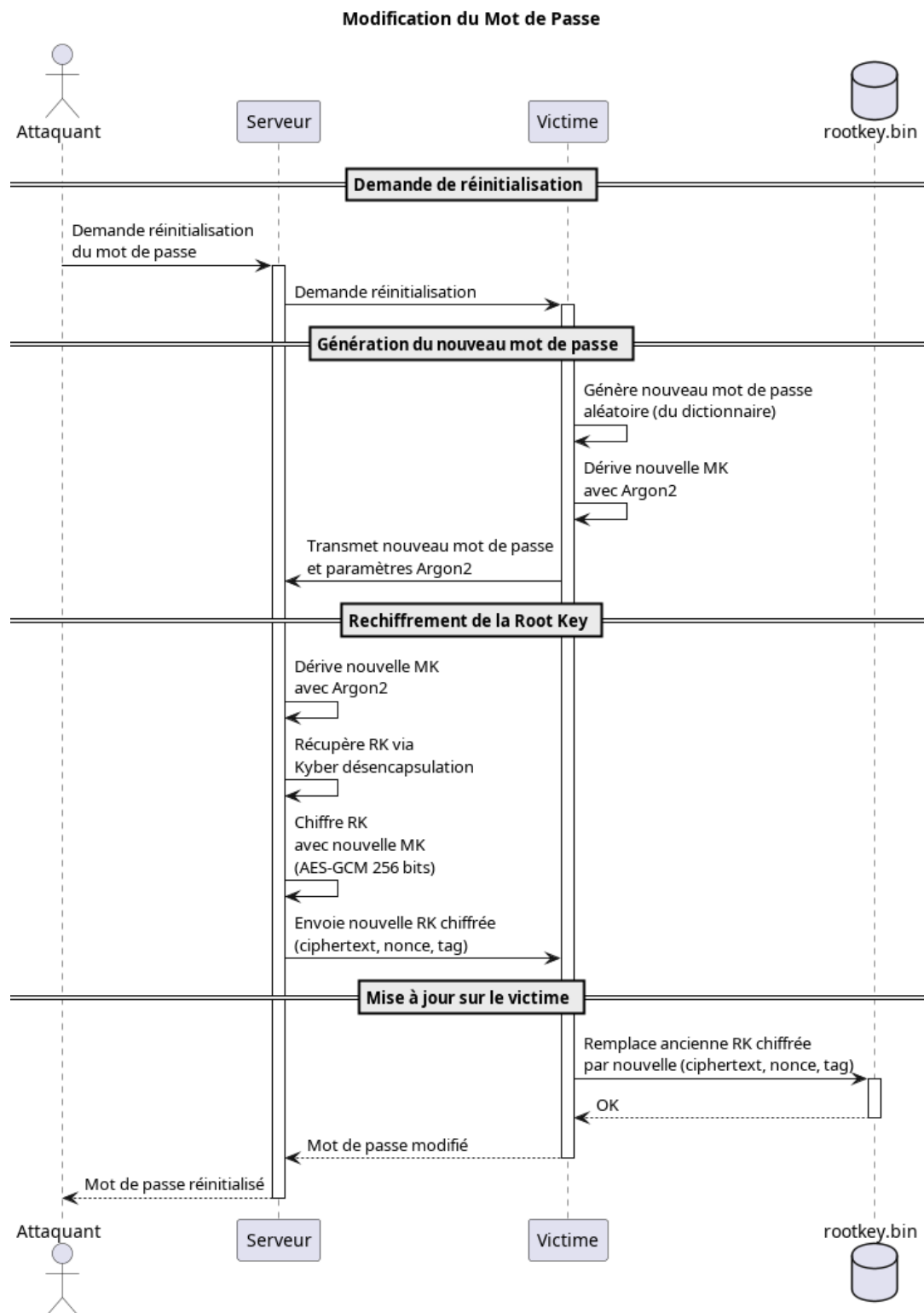


Fig. 6. – Changement de mot de passe sans rechiffrer les fichiers avec la méthode `change_password()`

4.5.1. Principe

La Root Key reste inchangée (elle protège tous les fichiers), seule la Master Key change (elle protège la Root Key). Il n'y aura donc pas besoin de rechiffrer les fichiers.

4.5.2. Étape 1 : Préparation du nouveau mot de passe

1. Générer un nouveau mot de passe :

- Le client génère un nouveau mot de passe aléatoire depuis la wordlist rockyou.txt.
- Génère un nouveau salt de 128 bits.
- Prépare les nouveaux paramètres Argon2.

2. Récupérer le `kyber_ciphertext` :

- Lit le fichier `rootkey.bin` existant et on extrait le `kyber_ciphertext`.

4.5.3. Étape 2 : Réencapsulation de la Root Key

Le client envoie une demande au serveur avec : Le nouveau password, le nouveau salt, les nouveaux paramètres Argon2 et le `kyber_ciphertext`

Le serveur va :

1. Dériver la nouvelle Master Key :

- Utilise Argon2id avec le nouveau password et le nouveau salt. Cela produit la nouvelle MK.

2. Récupérer la Root Key :

- Utilise sa clé secrète Kyber pour désencapsuler le `kyber_ciphertext`.
- Récupère la RK originale.

3. Réencapsuler avec la nouvelle MK :

- Encapsule la RK avec la nouvelle MK (AES-GCM). Ce qui produit un nouveau `wrapped_rk_ciphertext` (tag inclus en suffixe) et un nouveau `wrapped_rk_nonce`.
- Retourne ces nouvelles métadonnées au client.

4.5.4. Étape 3 : Mise à jour des métadonnées

1. Mettre à jour `rootkey.bin` :

- Le client remplace les anciennes métadonnées `wrapped_rk_ciphertext` et `wrapped_rk_nonce`.
- Conserve le même `kyber_ciphertext`.
- Met à jour le salt et les `argon2_params`.
- Sauvegarde le nouveau fichier `rootkey.bin`.

5. Implémentation technique

5.1. Architecture du code

L'implémentation a été réalisée en Python, avec une séparation claire entre le client et le serveur.

5.1.1. Structure des fichiers

config.py : Configuration et constantes

- Définit toutes les tailles de clés (`KEY_SIZE` = 32 pour 256 bits)
- Paramètres Argon2 : `time_cost=2`, `memory_cost=65536`, `parallelism=4`
- Extensions des fichiers générés : `KEY_EXTENSION` = ".key", `ROOT_KEY_EXTENSION` = ".root_key", `ENC_EXTENSION` = ".enc"
- Nom du fichier racine : `rootkey.bin`
- Listes d'exclusion pour protéger le code source du chiffrement

crypto_utils.py : Opérations cryptographiques

Ce module fournit des wrappers autour des bibliothèques cryptographiques.

wordlist.py : Génération de mots de passe

- Charge et filtre `rockyou.txt` (8-15 caractères)
- Maintient un cache de 50000 mots en mémoire
- Sélectionne un seul mot de 8-15 caractères (ex: « password123 »)
- Utilise `secrets.choice()` pour une sélection cryptographiquement sûre

server.py : Serveur de gestion des clés

La classe `RansomwareServer` gère :

- Génération de la paire de clés Kyber (publique/secrète)
- Génération du mot de passe aléatoire et du salt
- Stockage sécurisé de la clé secrète Kyber en mémoire
- Méthode `initialize_server()` : retourne password, salt, paramètres Argon2 et clé publique Kyber
- Méthode `request_full_decryption_credentials()` : fournit les credentials pour déchiffrement complet
- Méthode `decrypt_file_key()` : reçoit un fichier `.root_key`, reconstruit la RK via Kyber et désencapsule la clé (déchiffrement individuel)
- Méthode `change_password()` : permet de changer le mot de passe sans rechiffrer les fichiers

client.py : Client de chiffrement/déchiffrement

La classe `RansomwareClient` implémente les méthodes principales :

- `encrypt_directory()` : lance le chiffrement hiérarchique récursif via `_encrypt_dir_recursive()`
- `decrypt_all()` : déchiffrement hiérarchique récursif via `_decrypt_dir_recursive()`
- `decrypt_file()` : déchiffrement individuel d'un fichier via le serveur (envoie le `.root_key`)
- `change_password()` : changement de mot de passe sans rechiffrement
- `_encrypt_dir_recursive()` : gère la hiérarchie des clés (folder_key / file_key / parent_key)
- `_encrypt_file_to_enc()` : chiffre un fichier et le sauvegarde en `.enc`
- `_save_wrapped_key()` / `_load_wrapped_key()` : sérialisation/désérialisation des clés encapsulées
- `_should_skip_file()` / `_should_skip_dir()` : filtrage des fichiers/dossiers exclus

main.py : Interface utilisateur

Ce module fournit un menu interactif avec 5 options qui permet de tester toutes les fonctionnalités du ransomware. Il gère les entrées utilisateur avec des valeurs par défaut pour simplifier les tests et affiche des messages d'erreur en cas de problème.

5.2. Bibliothèques cryptographiques utilisées

cryptography : Chiffrement symétrique

- Module `AESGCM` pour le chiffrement authentifié
- Utilisé pour chiffrer les fichiers ET encapsuler toutes les clés
- Bibliothèque mature et audité, largement utilisée en production

argon2 : Dérivation de clés

- Module `argon2.low_level.hash_secret_raw` pour Argon2id
- Résistant aux attaques par GPU, ASIC et tables arc-en-ciel
- Paramètres calibrés pour 0.5s de calcul sur machine moderne

pqcrypto : Cryptographie post-quantique

- Implémentation de ML-KEM-1024 (CRYSTALS-Kyber niveau 5)
- Utilisé uniquement pour générer et échanger la Root Key
- Bibliothèque conforme aux spécifications NIST

secrets : Génération aléatoire sécurisée

- Utilisé pour toutes les générations aléatoires (clés, nonces, salts)
- Utilise le CSPRNG du système d'exploitation

5.3. Choix d'implémentation

L'architecture possède une séparation client/serveur. Dans ce projet, la communication est simulée localement grâce à des appels de fonctions Python. Le serveur conserve la clé secrète Kyber en mémoire pendant toute la session, tandis que le client ne manipule jamais directement cette clé secrète.

Concernant la gestion des métadonnées, le système utilise un format JSON avec encodage base64. Chaque fichier chiffré produit deux ou trois artefacts : un fichier `.enc` contenant le nonce (12 octets) concaténé au ciphertext avec tag (16 octets en suffixe), un fichier `.key` contenant la clé encapsulée avec la clé parente, et un fichier `.root_key` (pour les items hors niveau racine) contenant la clé encapsulée avec la Root Key. Les dossiers génèrent également des fichiers `.key` (et `.root_key` si hors racine). Le fichier `rootkey.bin` contient la Root Key encapsulée avec la MK, le ciphertext Kyber, le salt et les paramètres Argon2.

Chaque fichier utilise une clé unique générée aléatoirement, dans un but d'isolation des fichiers. Ainsi, si une clé de fichier est compromise, les autres fichiers restent protégés.

L'utilisation de wrappers autour des bibliothèques cryptographiques améliore le maintien du code. Toutes les opérations cryptographiques utilisent le module `crypto_utils.py`.

Enfin cette architecture est résistante aux attaques quantiques grâce à l'utilisation de CRYSTALS-Kyber (ML-KEM-1024) pour la génération et l'échange de la Root Key. Les ordinateurs quantiques, grâce à l'algorithme de Shor, peuvent casser les schémas cryptographiques asymétriques, comme RSA ou encore Diffie-Hellman. CRYSTALS-Kyber, utilise le problème de Learning With Errors (LWE) sur les lattices, ce qui reste difficile même pour les ordinateurs quantiques. Pour le chiffrement symétrique, AES-256 reste sécurisé face aux attaques quantiques. L'algorithme de Grover permet une accélération, ce qui réduit la sécurité de 256 bits à 128 bits. Cela reste suffisant. Argon2id conserve sa robustesse car les algorithmes quantiques n'offrent pas d'avantage significatif contre les fonctions de hachage cryptographiques basées sur des opérations itératives.

6. Tests et validation**6.1. Structure des tests**

Le fichier `app/test/test.py` valide les fonctionnalités principales :

Test 1 - Chiffrement hiérarchique :

- Chiffrement récursif du dossier `dossier_0`
- Vérification de la création de `rootkey.bin`
- Vérification de la création des fichiers `.enc`, `.key` pour les items racine
- Vérification de la création des fichiers `.root_key` pour les items dans les sous-dossiers

Test 2 - Déchiffrement complet :

- Déchiffrement de tous les fichiers via `decrypt_all()`
- Vérification de la restauration du contenu original à chaque niveau de l'arborescence
- Suppression des fichiers `.enc`, `.key` et `.root_key`

Test 3 - Changement de mot de passe :

- Génération d'un nouveau mot de passe
- Mise à jour de `rootkey.bin`
- Vérification que les fichiers `.enc` restent chiffrés

Test 4 - Déchiffrement d'un fichier spécifique :

- Rechiffrement du dossier
- Déchiffrement d'un seul fichier via `decrypt_file()` (utilise `.root_key` → serveur)
- Vérification que les autres fichiers restent chiffrés

7. Fonctionnalités bonus

Ce projet implémente toutes les fonctionnalités de base demandées dans la consigne. Une fonctionnalité supplémentaire a été ajoutée :

7.1. Chiffrement récursif hiérarchique de dossiers

Le chiffrement récursif (`encrypt_directory()`) et le déchiffrement récursif (`decrypt_all()`) permettent de traiter une arborescence complète de fichiers tout en maintenant une hiérarchie de clés cryptographiques.

7.1.1. Fonctionnement

Le chiffrement utilise une récursion via `_encrypt_dir_recursive(dir_path, parent_key, root_key, is_root_level)` à la place d'un `os.walk()` plat. Cela permet de propager la clé parente à chaque niveau de l'arborescence.

À chaque niveau de récursion :

1. **Snapshot des entrées** : `list(dir_path.iterdir())` est réalisé en début de fonction pour éviter d'itérer sur des fichiers créés pendant le traitement (`.key`, `.root_key`, `.enc`).
2. **Traitement des dossiers** : pour chaque sous-dossier non exclu, une `folder_key` est générée et l'encapsulation est effectuée avec `parent_key` (→ `.key`) et éventuellement avec `root_key` (→ `.root_key` si hors-racine). La récursion est lancée avec `folder_key` comme `parent_key`.
3. **Traitement des fichiers** : `_should_skip_file()` vérifie le nom et l'extension contre `EXCLUDED_FILES` et `EXCLUDED_EXTENSIONS` (structures `set`, vérification $O(1)$).

7.1.2. Complexité

Soient :

- D : nombre total de répertoires visités
- F : nombre total de fichiers à chiffrer
- S : taille totale en octets des fichiers à chiffrer

Opération	Complexité	— —	Traversal récursif	$O(D + F)$	Filtrage (<code>_should_skip_dir</code> , <code>_should_skip_file</code>)	$O(1)$ par appel (lookup <code>set</code>)	Génération clé + encapsulations par item	$O(1)$ (taille fixe AES-GCM)	Chiffrement d'un fichier (<code>_encrypt_file_to_enc</code>)	$O(f)$ proportionnel à sa taille	Total	$O(D + F + S)$
-----------	------------	-------	--------------------	------------	---	---	--	------------------------------	--	------------------------------------	--------------	----------------

En pratique, le terme dominant est $O(S)$: lire et chiffrer les octets de chaque fichier prend plus de temps que la traversal et les opérations cryptographiques sur les clés (taille fixe 32 octets). Le coût supplémentaire par rapport à une approche plate est l'encapsulation des `folder_key` et des `.root_key`, qui est négligeable (AES-GCM sur 32 octets).

8. Améliorations possibles

La robustesse du mot de passe n'est pas optimale au niveau de la sécurité. Le système génère un mot de passe unique de 8-15 caractères qui vient de la wordlist `rockyou.txt` et accepte tous les caractères incluant les symboles. Cependant, elle reste vulnérable aux attaques par dictionnaire car `rockyou.txt` contient des mots de passe couramment utilisés et compromis. L'entropie dépend de la taille du dictionnaire filtré (estimée entre 20-30 bits selon le nombre de mots disponibles de 8-15 caractères dans `rockyou.txt`), ce qui reste bien inférieur aux 128 bits de sécurité visés pour le reste du système. Une amélioration serait de générer des mots de passe vraiment aléatoires composés de caractères alphanumériques et symboles (exemple : `Kz9#mP2@xQ7!` = 71 bits d'entropie pour 12 caractères), plutôt que de se baser sur un dictionnaire de mots de passe connus.

Au niveau des performances, l'implémentation pourrait être optimisée en utilisant le chiffrement parallèle des fichiers via multiprocessing, permettant de traiter plusieurs fichiers simultanément. Pour les fichiers volumineux, une approche de chiffrement par blocs permettrait d'améliorer les performances.

Du côté du réseau, le système pourrait utiliser un réseau HTTPS avec le serveur pour simuler un vrai ransomware.

9. Conclusion

Ce projet démontre l'implémentation d'un ransomware en utilisant la cryptographie post-quantique. L'utilisation de CRYSTALS-Kyber (ML-KEM-1024) pour la génération de la Root Key garantit une résistance aux attaques quantiques futures, conformément aux recommandations du NIST.

10. Utilisation de l'IA

Rédaction du rapport

La rédaction de ce rapport a bénéficié de l'assistance d'intelligences artificielles, notamment GPT-5.2 et Claude Sonnet 4.5. Ces outils ont principalement été utilisés pour :

- La reformulation de phrases afin d'améliorer la clarté et la qualité du texte.
- La structuration de certains paragraphes pour une meilleure cohérence.
- La vérification des termes techniques.
- La correction et ajouts d'éléments sur les schémas explicatifs.

Cette aide m'a permis de maintenir une certaine qualité dans la rédaction tout en respectant l'aspect technique et les objectifs pédagogiques du projet. L'IA n'a pas écrit le rapport à ma place, mais elle a été utilisée comme un outil de soutien à la rédaction.

Assistance au développement

L'IA a également été utilisée lors du développement pour :

- Le débogage afin d'identifier et résoudre des erreurs de code ou d'exécution
- Optimiser et améliorer certaines fonctions.
- Résoudre des problèmes techniques qui ont demandé une assistance lors de blocages sur des configurations spécifiques.
- La compréhension des bibliothèques et la clarification de la documentation des technologies utilisées.

L'usage de l'IA a donc servi d'assistant de débogage et de documentation, pour faciliter la compréhension de certains messages d'erreur et accélérer le processus de développement.